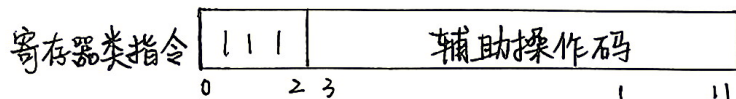
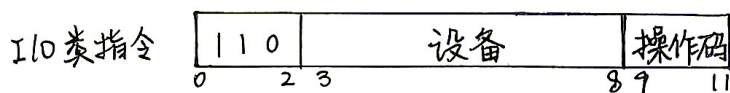
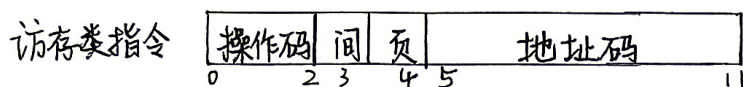


指令格式举例

设计指令格式时应考虑的各种因素：

- ① 指令系统的兼容性(向上兼容)
- ② 操作类型：指令个数及操作的难易程度
- ③ 数据类型：确定哪些数据类型可参与操作
- ④ 指令格式：指令字长是否固定；操作码位数、是否采用扩展操作码技术、地址码位数、地址个数、寻址方式类型
- ⑤ 寻址方式：指令寻址、操作数寻址
- ⑥ 寄存器个数：寄存器的多少直接影响指令的执行时间

1) PDP-8 指令字长固定12位

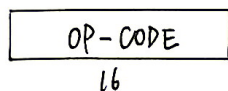


扩展操作码技术

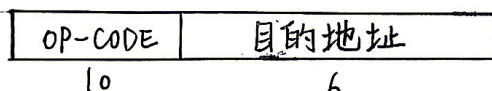
操作数由辅助操作码指定

2) PDP-11 指令字长有16位、32位、48位3种

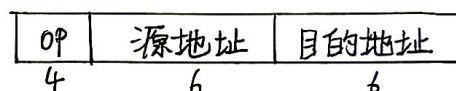
扩展操作码技术



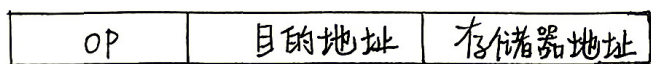
零地址(16位)



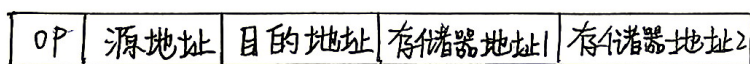
一地址(16位)



二地址 R-R (16位)

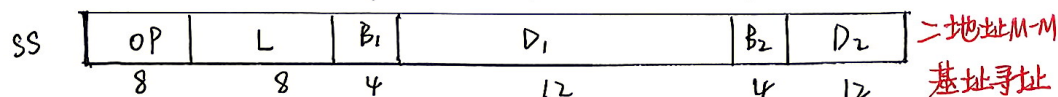
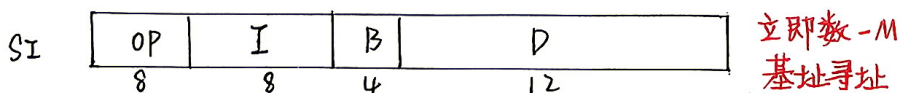
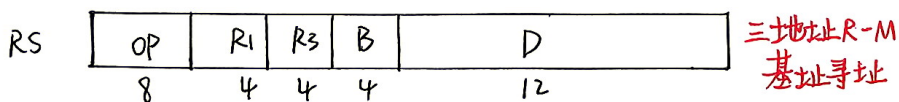
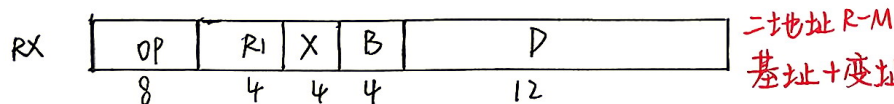
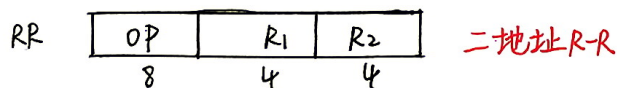


二地址 R-M (32位)



二地址 M-M (48位)

3) IBM 360 8位OP, 最多可表示256条指令



4) Intel 8086

① 指令字长 1~6 个字节

INC AX 1字节

MOV WORD PTR [0204], 0138H 6字节

② 地址格式

零地址 NOP 1字节

一地址 CALL 段间调用 5字节

CALL 段间调用 3字节

二地址 ADD AX, BX 2字节 寄存器-寄存器

ADD AX, 3048H 3字节 寄存器-立即数

ADD AX, [3048H] 4字节 寄存器-存储器

题目中若无特殊说明,一般地,机器字长 = 指令字长

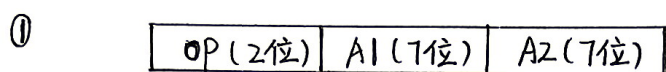
例. 某机器字长16位,存储器直接寻址空间是128字,变址时位移量为-64 ~ +63,
 16个通用寄存器均可作变址寄存器. 设计一套指令系统格式,满足下列寻址类型的要求:

寄存器编号4位

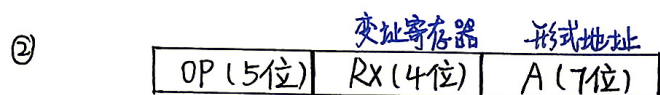
- ① 直接寻址的二地址指令3条
- ② 变址寻址的一地址指令6条
- ③ 寄存器寻址的二地址指令8条
- ④ 直接寻址的一地址指令12条
- ⑤ 零地址指令32条

★ 操作码扩展技术

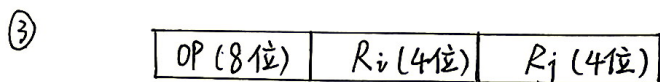
试问还有多少种代码未用? 若安排寄存器寻址的一地址指令,还能容纳多少条?



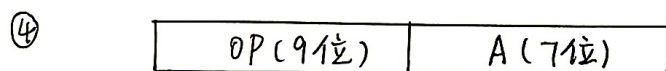
选00,01,10为操作码(3个),剩余11(1个)预留给操作码扩展窗口



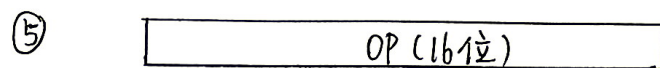
选11000 ~ 11101为操作码(16个),剩余11110,11111(2个)预留给操作码扩展窗口



选11110000 ~ 11110111为操作码(8个),剩余11111000 ~ 11111111(8个)预留给操作码扩展窗口



选111110000 ~ 111111011为操作码(12个),剩余111111100 ~ 111111111(4个)预留给操作码扩展窗口



选1111110000000000 ~ 1111110000111111为操作码(32个),

剩余 $512 - 32 = 480$ 条代码未用,若安排寄存器寻址的一地址指令,

除去末4位为寄存器地址外,还可容纳30条该类指令.

$$2^4 = 16$$

$$\frac{480}{16}$$

例. 设某机配有基址寄存器和变址寄存器, 采用一地址的指令系统, 允许直接和间接寻址, 且指令字长、机器字长和存储字长均为16位.

① 若采用单字长指令, 共能完成105种操作, 则指令可直接寻址的范围是多少? 画出其指令格式并说明各字段的含义.

一次间接寻址的寻址范围是多少?
有效地址长度并非指令中地址长度, 而是存储字长

② 若存储器字长不变, 采用何种方法直接访问容量为16MB的主存?

$8M \times 16$ 位

地址要23位.

解: ①

OP (7位)	M (2位)	AD (7位)
---------	--------	---------

105种操作, 操作码是7位

4种寻址方式, 寻址方式是2位

剩下形式地址位7位, 可直接寻址128个字

一次间址时, 地址长度是16位, 可寻址 $2^{16} = 65536$ 个字

②

OP (7位)	M (2位)	AD (23位)
---------	--------	----------

直接访问16MB的主存, 相当于8M字(字长16位),

故直接地址需要23位, 故必须采用双字长指令.

例. 某模型机共有64种操作, 操作码位数固定, 且具有以下特点:

① 采用一地址或二地址格式

② 有寄存器寻址、直接寻址、相对寻址(位移量为-128+127)三种寻址方式

③ 有16个通用寄存器、算术运算和逻辑运算的操作数均在寄存器中, 结果也在寄存器中

④ 取数/存数指令在通用寄存器和存储器之间传送数据

⑤ 存储容量为1MB, 按字节编址.

要求设计算逻指令、取数/存数指令和相对转移指令的格式, 并简述理由.

解:

① 算逻指令格式为RR型, 取单字长16位

OP (6位)	M (2位)	Ri (4位)	Rj (4位)
---------	--------	---------	---------

② 取数/存数指令格式为RS型, 32位长度

OP (6位)	M (2位)	Ri (4位)	A (20位)
---------	--------	---------	---------

③ 相对转移指令为一地址格式, 取单字长16位

OP (6位)	M (2位)	A (8位)
---------	--------	--------

例. 设某机器共能完成110种操作, CPU有8个通用寄存器(16位), 主存容量为4M字, 采用寄存器-存储器型指令.

寄存器-存储器型指令. $\xrightarrow{R}$ 操作码7位 $\xrightarrow{S}$ 编号3位 $\xrightarrow{}$ 寻址要2根地址线

$\xrightarrow{}$ 多1个字就是多1个位!

- ① 欲使指令可直接访问主存的任一地址, 指令长度应取多少? 画出指令格式. 32 位 2^n 个字就是要n个位
- ② 若在上述设计的指令字中设置一寻址特征位X, 且X=1表示某寄存器作基址寄存器, 表示! 画出指令格式. 试问基址寻址可否访问主存的任一单元? 为什么?

若不能, 提出一种方案, 使其可以访问主存的任一位置.

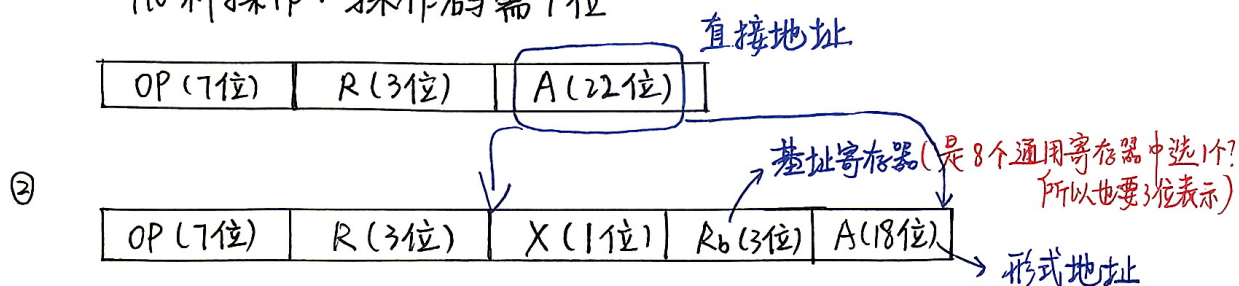
- ③ 若主存容量扩大到4G字, 且存储字长等于指令字长, 则在不改变上述硬件结构的前提下, 可采用什么方法使指令可访问存储器的任一位置? 把地址放在存储器中, 一次问址 (2次就没这么大了)

解:

- ① 4M字存储器, 需要22位地址

8个通用寄存器: 寄存器编号需3位

110种操作: 操作码需7位



$$\because 2^{16} + 2^{18} < 2^{22}$$

基址寄存器长度16位. 位移量寻址空间

其寻址空间 2^{16}

$\therefore$ 不能覆盖4M字寻址空间

解决方法: 将R_b的内容左移6位, 低位补0, 形成22位基地址,

然后与形式地址相加, 所得的有效地址即可访问4M字存储器的任一单元.

- ③ $\because 4G = 2^{32}$ 且存储字长 = 指令字长. $\rightarrow$ 需32位地址, 才有 2^{32} 的寻址空间

一个存储单元长度32位, 把地址放存储器中就可提供32位有效地址.

采用一次间接寻址可以访问任一存储单元

RISC 技术 (Reduced Instruction Set Computer)

CISC (Complex Instruction Set Computer)

80-20 规律

→ 控制器占芯片面积 60%

典型程序中 80% 的语句仅使用处理机中 20% 的指令。

执行频度高的简单指令因复杂指令的存在, 执行速度无法提高。

能否用 20% 的简单指令组合不常用的 80% 的指令功能?

由此, RISC 技术产生 (3 阶段)

芯片

VLSI 芯片

超大规模集成电路

RISC 由于设计的指令条数有限, 只需较小空间就可实现逻辑控制电路, 更多空间可以用来增强处理机性能或使处理器功能多样化。

RISC 的主要特征

- 选用使用 频度较高 的一些 简单指令, 复杂指令的功能由简单指令来组合
- 指令长度固定、指令格式种类少、寻址方式少
- 只有 LOAD/STORE 指令访存
(取数) (存数)
- CPU 中有多个通用寄存器
- 采用 流水技术, 一个时钟周期内完成一条指令
- 采用 组合逻辑 实现控制器
- 采用优化的编译程序

指令比较简单且规整, 指令间关联较小, 容易找出能够并行执行的指令, 安排在流水线中, 可提升机器的程序执行速度

CISC 的主要特征

- 系统指令复杂庞大, 各种指令使用频度相差大
- 指令长度不固定、指令格式种类多、寻址方式多
- 访存指令不受限制
- CPU 中设有专用寄存器
- 大多数指令需要多个时钟周期执行完毕

即使采用流水线作业, 速度也以最慢的那条指令为准

- 采用 微程序 控制器

→ 微程序其实是一种硬件, 采用程序设计的思想。

把指令执行过程写成微小操作。把微程序存储在控制器

[取、执行] 速度慢。

- 难以用优化编译生成高效的且的代码

→ 错综复杂

RISC 与 CISC 的比较：

① RISC 更能充分利用 VLSI 芯片的面积

② RISC 更能提高计算机运算速度

指令数、指令格式、寻址方式少，通用寄存器多，
采用组合逻辑，便于实现指令流水。

③ RISC 便于设计，可降低成本，提高可靠性

④ RISC 有利于编译程序代码优化

⑤ RISC 不易实现指令系统兼容（已是底层最简单的了，不能互相识别）

一个较完善的指令系统应有数据处理、数据存贮、数据传送、程序控制四大类指令

寻址方式：
指令寻址方式：顺序寻址方式、跳跃寻址方式
数据寻址方式：
隐含寻址（0次）、直接寻址（1次）、立即寻址（0次）、间接寻址（1次）、寄存器寻址（0次）、寄存器间接寻址（1次）、
相对寻址（1次）、基址寻址（1次）、变址寻址（1次）、堆栈寻址（1次）
形成指令地址或数据地址的方式

RISC 特点：

① 指令条数少

② 指令长度固定，指令格式和寻址方式种类少

③ 只有取数/存数指令访问存储器，其余指令操作均在寄存器间进行